

Functional-OO Assignment

CS F301 POPL Assignment | November 2025

By: Aryan Bethmangalkar (2023A7PS0433G) **Team:** Solo

Abstract

This project extends a from-scratch C++ list container library, made in the first assignment, with a small Functional-OO layer: template functions for functional operations (map, filter, reduce), utility aggregators, file I/O for reading and tokenizing plain text, basic search and sort algorithms, and a keyword-frequency analysis tool which is the primary deliverable for the assignment. The code is modular, tested, and accompanied by an interactive CLI demo and concise documentation.

Contents of this report

1. Quick start & prerequisites
 2. Detailed project structure
 3. Design philosophy and key decisions
 4. Module-by-module descriptions and API summary
 5. Keyword frequency works
 6. Tests and verification
 7. Implementation notes and notable code excerpts
 8. Known limitations and future work
 9. Design Log
 10. GenAI Usage Log
-

1. Quick start & prerequisites

Prerequisites

- macOS or Linux (POSIX APIs used for directory listing)
- g++ (C++11 or later)
- make

Build and run demo (recommended):

```
# Build everything and run the interactive demo
make all && ./demo
```

Run only tests (recommended for graders who prefer automated verification):

```

make tests
./test_functional
./test_fileio
./test_textanalysis
# or run all three sequentially
./test_functional && ./test_fileio && ./test_textanalysis

```

Notes

- All required demo inputs are in `test_data/` (keywords.txt + sample_texts/).
- The demo is menu-driven and interactive; option 4 runs the keyword frequency analysis (primary assignment requirement).

2. Detailed project structure

```

ListContainer/
├── Makefile
├── README.md
├── SUBMISSION_REPORT.md    # This file
├── design_log.md
├── genai_usage.md
├── .gitignore
├── common/
│   └── list_err.h          # error definitions and utility macros
├── containers/             # core, from-scratch data structures
│   ├── AbstractList.h
│   ├── SequentialList.h
│   ├── RestrictedSequentialList.h
│   ├── PriorityBased.h
│   ├── VectorList.h
│   ├── VectorList.cpp
│   ├── LinkedList.h
│   ├── LinkedList.cpp
│   ├── ListNode.h
│   ├── Stack.h
│   ├── Stack.cpp
│   ├── Queue.h
│   ├── Queue.cpp
│   ├── Deque.h
│   ├── Deque.cpp
│   ├── Heap.h
│   ├── Heap.cpp
│   └── PriorityQueue.h
├── functional/             # functional-00 layer (header-only templates)
│   ├── FunctionalOps.h     # map, filter, reduce, forEach
│   └── Aggregators.h       # sum, average, min, max, countIf, any/all

```

```

├── algorithms/                # small algorithm utilities
│   ├── Search.h              # linear, binary, predicate-based
│   └── Sort.h                # insertion, bubble, quicksort variants
(custom comparator)
├── io/                        # file and text utilities
│   ├── FileReader.h
│   ├── FileReader.cpp
│   ├── TextTokenizer.h
│   └── TextTokenizer.cpp
├── analysis/                  # assignment-specific analysis tools
│   ├── KeywordCounter.h
│   └── KeywordCounter.cpp
├── test_data/
│   ├── keywords.txt
│   └── sample_texts/ (doc1.txt, doc2.txt, doc3.txt,
programming_article.txt)
├── tests/
│   ├── test_functional.cpp
│   ├── test_fileio.cpp
│   └── test_textanalysis.cpp
├── demo.cpp                  # interactive CLI demonstration (menu-driven)
└── main.cpp                  # original demo / compatibility

```

3. Design philosophy and key decisions

High-level goals

- Separate WHAT (high-level specification) from HOW (implementation details).
- Keep the functional layer at the top (free template functions that express WHAT), and implement HOW using OO classes that manage resources and encapsulate behavior.
- Preserve and extend the from-scratch container implementations from the earlier assignment; avoid using standard library associative containers for the core assignment features (as required).

Key decisions

- Functional operations implemented as header-only free template functions (in `functional/FunctionalOps.h`) — this preserves the Open-Closed Principle and allows operating on any `AbstractList<T>`.
- For containers without random-access (`Stack`, `Queue`), `FunctionalOps` uses a fallback path (pop/push with temporary buffer) and an optimized path for `SequentialList` (which exposes `at()`). This provides generality with acceptable trade-offs.
- Keyword frequency uses `VectorList<KeywordFrequency>` (from-scratch) and a custom sort; chosen for clarity and to meet the "from-scratch" constraint.

- File I/O uses POSIX `dirent.h` for directory listing (documented in `genai_usage.md`); this is simple and reliable on the target platform (macOS/linux).

4. Module-by-module descriptions and API summary

This section gives a concise API-like summary so graders can quickly locate and test relevant pieces.

containers/

- `AbstractList` — pure abstract interface (push/pop/size/clear/name)
- `SequentialList` — adds index-based methods (`insert_at`, `erase_at`, `at`)
- `VectorList` — dynamic array implementation. Important methods: `push`, `pop`, `insert_at`, `erase_at`, `at`, `size`.
 - Also implements copy constructor and copy-assignment operator (deep copy) to enforce Rule of Three.
- `LinkedList` — node-based sequential container with similar API.
- `Stack/Queue/Deque` — adapters around `VectorList/LinkedList` for restricted behavior.
- `Heap/PriorityQueue` — priority-based containers built on top of `VectorList`.

functional/

- `map<AbstractList, F>` → returns a `VectorList<U>` containing `transform(x)` for each `x`
- `filter<AbstractList, P>` → returns `VectorList<T>` with elements satisfying predicate
- `reduce<AbstractList, Combiner>` → accumulates values using combiner and initial value
- `forEach<AbstractList, Action>` → applies side-effect action to each element
- Aggregators: `sum`, `average`, `min`, `max`, `countIf`, `any`, `all`

algorithms/

- `Search.h`: `indexOf`, `contains`, `findAll` (supports predicate)
- `Sort.h`: `insertionSort`, `bubbleSort`, `quickSort` variants that accept comparator functions

io/

- `FileReader`
 - `VectorList<string> readDirectory(const string &path)` — returns a list of file paths
 - `string readFileToString(const string &path)`
- `TextTokenizer`
 - `VectorList<string> tokenize(const string &text, bool toLower=true, bool removePunct=true)`

analysis/

- `KeywordCounter`
 - `VectorList<KeywordFrequency> analyzeFiles(VectorList<string> keywords, VectorList<string> filepaths)`
 - Supports case-sensitive or case-insensitive modes via configuration
 - Tracks statistics: `totalWordsProcessed`, `keywordsFound`

5. Keyword frequency analysis

Algorithm (high-level)

1. Load keywords from `keywords.txt` into a `VectorList<string>`.
2. Initialize a `VectorList<KeywordFrequency>` with each keyword and count=0.
3. For each data file in the selected directory:
 - Read file content with `FileReader`.
 - Tokenize using `TextTokenizer` (case normalization and punctuation removal are optional settings).
 - For each token (word), scan the `KeywordFrequency` list to find a match. If found, increment its count.
4. When all files processed, sort `KeywordFrequency` list by count descending and present results.

Complexity and trade-offs

- This approach is $O(\text{totalWords} * \text{numKeywords})$ — $O(n*m)$ — not optimal for extremely large corpora but simple, easy to validate, and consistent with a "from-scratch" constraint (no hash tables).
- For larger data, a hash-table or trie would be significantly faster and could be added as an extension.

Sample expected output (from demo)

```

Loading keywords: keywords.txt (15 keywords)
Analyzing directory: test_data/sample_texts/ (4 files)
Files processed: 4
Total tokens: 1051
Top keywords:
  data      34
  search    18
  structure 12
  algorithm 10
  sort      7

```

6. Tests and verification

Test suite overview (located in `tests/`)

- `test_functional.cpp` — verifies `map`, `filter`, `reduce`, `forEach`, and aggregators (6 tests)
- `test_fileio.cpp` — verifies `FileReader::readFileToString`, `readDirectory`, and `TextTokenizer` (3 tests)
- `test_textanalysis.cpp` — verifies `KeywordCounter` correctness and case handling (3 tests)

How to run

```

make tests
./test_functional
./test_fileio
./test_textanalysis

```

Expected outcome

- All tests should pass. The Makefile contains targets to build each test binary individually and a combined `make tests` target.

Automated verification note

- The test binaries are small and run in < 1 second on a typical laptop; they print PASS/FAIL messages for quick evaluation.

7. Implementation notes and notable code excerpts

Memory management

- `VectorList` manages a raw dynamic array; originally the class had a destructor but no copy-constructor/assignment operator, which caused a double-free bug when a temporary or copy was destroyed. The fix:
 - Implemented `VectorList(const VectorList<T>& other)` which allocates new memory and copies elements.
 - Implemented `VectorList<T>& operator=(const VectorList<T>& other)` with self-assignment check and deep copy.

Functional operations design

- `map/filter/reduce` are header-only template functions that accept `std::function` or callable objects (lambdas). They return a new `VectorList` to preserve immutability at the operation level.

File I/O and tokenization

- `FileReader::readDirectory` uses POSIX `opendir/readdir` and filters for regular files (optionally `.txt`).
- `TextTokenizer` strips punctuation and optionally lowercases tokens for case-insensitive matching.

Sorting and presentation

- `KeywordFrequency` is a small struct (`string keyword; int count;`) with comparison operators defined to allow sorting by count descending.

8. Known limitations and suggested future work

Limitations

- Keyword counting uses $O(n*m)$ scanning; not suitable for very large corpora.
- File I/O relies on POSIX `dirent.h` (not natively portable to Windows without compatibility layers).
- Sorting implementations are simple and may not be as optimized as `std::sort`.

Suggested improvements

- Add a `HashMap`-based `KeywordCounter` for large-scale performance.

- Replace POSIX APIs with C++17 `std::filesystem` for cross-platform portability.
- Add more unit tests and fuzz tests for tokenization edge cases.

9. Design Log

The complete design and development log is presented here for evaluators who prefer a single-file report. (Timestamps are local to the development environment.)

Entry 1: Assignment Analysis (23/11/2025 14:30)

Requirements Identified:

- Extend list containers with functional programming (map, filter, reduce)
- File I/O for reading and tokenizing text files
- Keyword frequency analysis from multiple files
- Menu-driven interface for demonstration

Design Philosophy:

- Focus on WHAT (specification) before HOW (implementation)
- Free template functions for functional operations (not modifying base classes)
- Composition for I/O modules (FileReader, TextTokenizer)

Entry 2: Key Design Decisions (23/11/2025 14:45)

1. Functional Operations as Free Functions

- Why: Maintains Open-Closed Principle, works with any `AbstractList`
- Implementation: Template functions using `std::function` for lambda support

2. Interactive Menu-Driven CLI

- Why: Easy to demonstrate and test, clear for evaluators
- Alternative considered: Command-line args (less interactive)

3. Keyword Counting with `VectorList`

- Why: Maintains from-scratch requirement, simple and testable

4. OO in Implementation, Functional at Top Level

- How (implementation): Classes with encapsulation (FileReader, TextTokenizer, KeywordCounter)
- What (top level): Functional composition of operations (map → filter → reduce)
- Pattern: Adapter pattern for containers, Composition pattern for I/O modules

Entry 3: Module Architecture (23/11/2026 15:30)

Directory Structure:

```
functional/ - map, filter, reduce, aggregators
io/         - FileReader, TextTokenizer
analysis/   - KeywordCounter (primary feature)
algorithms/ - Search and sort
test_data/  - Sample keywords and text files
tests/      - Comprehensive test suite
```

Dependency Chain: containers → functional → algorithms → io → analysis → demo

Entry 4: Implementation Phase

Implemented Components (23/11/2025 16:00):

- Functional operations: map, filter, reduce, forEach, aggregators
- File I/O: Read files/directories, tokenize with punctuation removal

Implemented Components (25/11/2025 20:00):

- Keyword analysis: Load keywords, count frequencies, sort descending
- Search/Sort: Linear, binary search; QuickSort, bubble, insertion sort

Implemented Components (26/11/2025 9:00):

- Interactive demo with 6 menu sections
- Test suite: 15 tests covering all modules

Time Spent: ~8 hours total

Entry 5: Critical Bug Fix (26/11/2025 11:30)

Problem: Double free error - `malloc: double free for ptr 0x151810600`

Root Cause: VectorList violated Rule of Three (had destructor, no copy constructor/assignment operator)

Solution: Implemented deep copy semantics:

- Copy constructor allocates new memory, copies elements
- Assignment operator with self-assignment check

Result: All 15 tests passing, no memory errors

10. GenAI Usage Log

Below is the complete GenAI usage log documenting prompts, key suggestions, incorporations and outcomes.

Interaction 1: Architecture Design

Tool: GitHub Copilot

Date: 23/11/2025 14:00

Prompt: "Analyze PoPL assignment and create implementation plan with directory structure and testing strategy"

Key Suggestions:

- Modular structure: functional/, io/, analysis/, algorithms/
- Free template functions for functional ops (not methods in container classes)
- Interactive menu-driven CLI for easy demonstration

Insight: Functional programming in C++ works best with free functions using templates and `std::function`. Keeps functional paradigm separate from OO structure.

Incorporation: Adopted modular structure, implemented all functional operations as free template functions.

Result: Successful - Clean, maintainable architecture

Interaction 2: Handling Non-Sequential Containers

Tool: GitHub Copilot

Date: 23/11/2025 14:30

Prompt: "How to implement map/filter/reduce for containers without random access like Stack?"

Suggestion: Use `dynamic_cast` to check container type, use efficient path for `SequentialList` (has `at()`), fall back to `pop/push` for others.

Insight: Generic algorithms need multiple implementation paths based on container capabilities.

Incorporation: Added runtime type checking in `FunctionalOps.h` with conditional logic.

Result: Works for all container types

Interaction 3: File I/O Implementation

Tool: GitHub Copilot Autocomplete

Date: 23/11/2025 15:00

Suggestions:

- Use `<fstream>` for file reading
- Use `<dirent.h>` for directory listing (POSIX)
- Use `<sys/stat.h>` for file existence checks

Trade-off: POSIX APIs work on macOS/Linux but not Windows. Acceptable for this assignment's platform.

Incorporation: Implemented `FileReader` using POSIX APIs with proper error handling.

Result: Works perfectly on macOS

Interaction 4: Sorting Algorithm Choice

Tool: GitHub Copilot

Date: 25/11/2025 19:00

Prompt: "Best way to sort VectorList with custom comparator without std::sort?"

Warning: Using `erase_at/insert_at` in sorting is inefficient ($O(n)$ per operation). Suggested simpler algorithms like insertion sort.

Insight: With constrained API, simpler algorithms (insertion sort) are more reliable than complex ones (quicksort).

Incorporation: Implemented insertion sort and bubble sort for sorting operations.

Result: Works but not optimal - acceptable for assignment scope

Interaction 5: Keyword Frequency Algorithm

Tool: GitHub Copilot

Date: 25/11/2025 20:30

Prompt: "Implement keyword frequency counting without std::map"

Suggestion: Use `VectorList<KeywordFrequency>` struct. For each word: search list, increment if found, add if not found, then sort by count.

Trade-off: $O(nmk)$ complexity vs $O(n*m)$ with hash table. Simple and meets "from-scratch" requirement.

Incorporation: Implemented exactly as suggested in KeywordCounter.

Result: Works correctly for moderate-sized files

Interaction 6: Memory Management Bug

Tool: GitHub Copilot

Date: 26/11/2025 11:00

Prompt: "Double free error in VectorList - how to implement Rule of Three properly?"

Suggestions:

- Add copy constructor with deep copy (allocate new array, copy elements)
- Add assignment operator with self-assignment check
- Both need to prevent shared pointers between objects

Insight: C++ Rule of Three - if class manages resources (has destructor), must define copy constructor and assignment operator.

Incorporation: Implemented deep copy semantics in VectorList.h/cpp.

Result: All tests passing, no memory errors

Summary of GenAI usage

Total Interactions: 6 major interactions

Success Rate: 5 fully successful, 1 partially successful

Most Valuable Insight: Free template functions for functional programming in C++ maintain separation of concerns and work with any AbstractList type.

Key Learnings:

- Design decisions documented before implementation
 - Simple solutions often better with API constraints
 - GenAI excellent for architecture guidance, human judgment needed for trade-offs
 - Rule of Three is critical for resource-managing classes
-

Conclusion

Verification: the project includes an automated test suite (`tests/`) and an interactive `demo` binary; after the Rule-of-Three fix to `VectorList` all tests pass on the development machine. To reproduce locally, build and run the demo or tests as shown in the Quick start section. There is also a demo video called `VideoDemo.mp4`.

Where to look next: key files are `functional/FunctionalOps.h` (functional primitives), `containers/VectorList.h` (container implementation and copy semantics), `analysis/KeywordCounter.cpp` (counting and sorting), and `demo.cpp` (interactive demonstration). For quick verification, run the tests and the demo and inspect `test_data/` for sample inputs.

This assignment implements the Functional-OO extension and the keyword frequency analysis required by the assignment. The implementation preserves the from-scratch constraint for core containers, provides a small header-only functional layer (map/filter/reduce), includes file I/O and tokenization utilities, and a tested `KeywordCounter` tool that produces the required ranked keyword output.